Chapter IV

Exercise 00 : ft_strcpy

	Exercise 00
ft_strcpy	
Turn-in directory: *ex*00/	
Files to turn in: `ft_strcpy.c`	
Allowed functions: `None`	

- Reproduce the behavior of the function `strcpy` (man strcpy).
- Here is how it should be prototyped:

```
char        *ft_strcpy(char *dest, char *src);
```

Chapter V

Exercise 01 : ft_strncpy

	Exercise 01
ft_strncpy	
Turn-in directory: *ex*01/	
Files to turn in: `ft_strncpy.c`	
Allowed functions: `None`	

- Reproduce the behavior of the function `strncpy` (man strncpy).
- Here is how it should be prototyped:

```
char        *ft_strncpy(char *dest, char *src, unsigned int n);
```

Chapter VI

Exercise 02 : ft_str_is_alpha

	Exercise 02
	ft_str_is_alpha
Turn-in directory: *ex*02/	
Files to turn in: `ft_str_is_alpha.c`	
Allowed functions: `None`	

- Create a function that returns 1 if the given string contains only alphabetical characters and 0 if it contains any other character.
- Here is how it should be prototyped:

```
int        ft_str_is_alpha(char *str);
```

- It should return 1 if `str` is empty.

Chapter VII

Exercise 03 : ft_str_is_numeric

	Exercise 03
ft_str_is_numeric	
Turn-in directory: $ex03/$	
Files to turn in: `ft_str_is_numeric.c`	
Allowed functions: `None`	

- Create a function that returns 1 if the given string contains only digits and 0 if it contains any other character.
- Here is how it should be prototyped:

```
int        ft_str_is_numeric(char *str);
```

- It should return 1 if `str` is empty.

Chapter VIII

Exercise 04 : ft_str_is_lowercase

	Exercise 04
ft_str_is_lowercase	
Turn-in directory: *ex*04/	
Files to turn in: `ft_str_is_lowercase.c`	
Allowed functions: `None`	

- Create a function that returns 1 if the given string contains only lowercase alphabetical characters and 0 if it contains any other character.
- Here is how it should be prototyped:

```
int		ft_str_is_lowercase(char *str);
```

- It should return 1 if `str` is empty.

Chapter IX

Exercise 05 : ft_str_is_uppercase

	Exercise 05
	ft_str_is_uppercase
Turn-in directory: $ex05/$	
Files to turn in: `ft_str_is_uppercase.c`	
Allowed functions: `None`	

- Create a function that returns 1 if the given string contains only uppercase alphabetical characters and 0 if it contains any other character.
- Here is how it should be prototyped:

```
int        ft_str_is_uppercase(char *str);
```

- It should return 1 if `str` is empty.

Chapter X

Exercise 06 : ft_str_is_printable

	Exercise 06
ft_str_is_printable	
Turn-in directory: *ex*06/	
Files to turn in: `ft_str_is_printable.c`	
Allowed functions: `None`	

- Create a function that returns 1 if the given string contains only printable characters and 0 if it contains any other character.
- Here is how it should be prototyped:

```
int        ft_str_is_printable(char *str);
```

- It should return 1 if `str` is empty.

Chapter XI

Exercise 07 : ft_strupcase

	Exercise 07
ft_strupcase	
Turn-in directory: $ex07/$	
Files to turn in: `ft_strupcase.c`	
Allowed functions: `None`	

- Create a function that converts every letter to uppercase.
- Here is how it should be prototyped:

```
char        *ft_strupcase(char *str);
```

- It should return `str`.

Chapter XII

Exercise 08 : ft_strlowcase

	Exercise 08
ft_strlowcase	
Turn-in directory: *ex*08/	
Files to turn in: `ft_strlowcase.c`	
Allowed functions: `None`	

- Create a function that converts every letter to lowercase.
- Here is how it should be prototyped:

```
char        *ft_strlowcase(char *str);
```

- It should return `str`.

Chapter XIII

Exercise 09 : ft_strcapitalize

	Exercise 09
ft_strcapitalize	
Turn-in directory: *ex*09/	
Files to turn in: `ft_strcapitalize.c`	
Allowed functions: `None`	

- Create a function that capitalizes the first letter of each word and converts all other letters to lowercase.
- A word is a sequence of alphanumeric characters.
- Here is how it should be prototyped:

```
char        *ft_strcapitalize(char *str);
```

- It should return `str`.
- For example:

```
hi, how are you? 42words forty-two; fifty+and+one
```

- Becomes:

```
Hi, How Are You? 42words Forty-Two; Fifty+And+One
```

Chapter XIV

Exercise 10 : ft_strlcpy

	Exercise 10
ft_strlcpy	
Turn-in directory: $ex10/$	
Files to turn in: `ft_strlcpy.c`	
Allowed functions: `None`	

- Reproduce the behavior of the function `strlcpy` (man strlcpy).
- Here is how it should be prototyped:

```
unsigned int ft_strlcpy(char *dest, char *src, unsigned int size);
```

Chapter XV

Exercise 11 : ft_putstr_non_printable

	Exercise 11
	ft_putstr_non_printable
Turn-in directory: $ex11/$	
Files to turn in: `ft_putstr_non_printable.c`	
Allowed functions: `write`	

- Create a function that displays a string of characters on screen. If this string contains non-printable characters, they must be displayed as lowercase hexadecimal values, preceded by a backslash.
- For example:

```
Hello\nHow are you?
```

- The function should display:

```
Hello\0aHow are you?
```

- Here is how it should be prototyped:

```
void        ft_putstr_non_printable(char *str);
```

Chapter XVI

Exercise 12 : ft_print_memory

	Exercise 12
ft_print_memory	
Turn-in directory: $ex12/$	
Files to turn in: `ft_print_memory.c`	
Allowed functions: `write`	

- Create a function that displays a memory area on screen.
- The display of this memory area should be divided into three "columns", separated by a space:
  - The hexadecimal address of the first character in the line, followed by a ':'.
  - The content in hexadecimal, with a space every two characters, and padded with spaces if necessary (see the example below).
  - The content in printable characters.
- If a character is non-printable, it should be replaced by a dot
- Each line should display sixteen characters.
- If **size** is equal to 0, nothing should be displayed.

- Example:

```
$> ./ft_print_memory
000000010a161f40: 426f 6e6a 6f75 7220 6c65 7320 616d 696e Bonjour les amin
000000010a161f50: 6368 6573 090a 0963 0720 6573 7420 666f ches...c. est fo
000000010a161f60: 7509 746f 7574 0963 6520 7175 206f 6e20 u.tout.ce qu on
000000010a161f70: 7065 7574 2066 6169 7265 2061 7665 6309 peut faire avec.
000000010a161f80: 0a09 7072 696e 745f 6d65 6d6f 7279 0a0a ..print_memory..
000000010a161f90: 0a09 6c6f 6c2e 6c6f 6c0a 2000           ..lol.lol. .
$> ./ft_print_memory | cat -te
0000000107ff9f40: 426f 6e6a 6f75 7220 6c65 7320 616d 696e Bonjour les amin$
0000000107ff9f50: 6368 6573 090a 0963 0720 6573 7420 666f ches...c. est fo$
0000000107ff9f60: 7509 746f 7574 0963 6520 7175 206f 6e20 u.tout.ce qu on $
0000000107ff9f70: 7065 7574 2066 6169 7265 2061 7665 6309 peut faire avec.$
0000000107ff9f80: 0a09 7072 696e 745f 6d65 6d6f 7279 0a0a ..print_memory..$
0000000107ff9f90: 0a09 6c6f 6c2e 6c6f 6c0a 2000           ..lol.lol. .$
$>
```

- Here is how it should be prototyped:

```
void        *ft_print_memory(void *addr, unsigned int size);
```

- It should return `addr`.